

FORENSIC ARTIFACT TRIAGE TOOLKIT

Especificação Técnica de Engenharia de Software e Roadmap Operacional de Sprints

27 de setembro de 2026

1. VISÃO GERAL DO PROJETO

O Forensic Artifact Triage Toolkit é uma ferramenta computacional avançada desenvolvida em linguagem Python, projetada especificamente para a resposta ágil a incidentes cibernéticos e triagem de sistemas operacionais potencialmente comprometidos. Sua finalidade operacional primária consiste na coleta automatizada, preservação estrita com integridade criptográfica contínua e análise direcionada de artefatos essenciais do ecossistema Microsoft Windows, culminando na emissão de relatórios de auditoria técnica e pericial plenamente válidos e reproduzíveis.

Do ponto de vista estratégico de arquitetura corporativa, esta ferramenta se posiciona como a camada de resposta a incidentes e perícia digital (Fase Respond) de um ecossistema modular e integrado de segurança da informação já composto por três soluções consolidadas: o pii-guard (focado na varredura técnica e detecção ativa de dados pessoais regulados e credenciais/segregados vazados), o compliance-audit (motor de inteligência regulatória encarregado de gerar Matrizes de Riscos, Políticas de Segurança da Informação — PSI, Declarações de Aplicabilidade — SOA e relatórios de auditorias simuladas) e o compliance-suite (painel centralizado de monitoramento contínuo de postura de conformidade).

O toolkit fecha o ciclo de vida da segurança cibernética organizacional estruturado na metodologia Start → Control → Assure → Respond. Enquanto as soluções preexistentes garantem as etapas preventivas de diagnóstico, governança de controles e sustentação de conformidade perante normas internacionais e marcos regulatórios, o toolkit atua com rigor científico no momento exato em que a violação ou o comprometimento se materializa, assegurando a contenção forense sem comprometer a idoneidade probatória dos fatos investigados.

2. OBJETIVOS E REQUISITOS GERAIS

A concepção e a implementação do software norteiam-se por metas técnicas e jurídicas indeclináveis, estruturadas nos seguintes objetivos fundamentais:

- Aquisição e Custódia com Garantia Criptográfica: Executar a extração de artefatos digitais voláteis e persistentes mantendo cadeia de custódia ininterrupta e

verificável, apoiada em funções de hashing seguro multialgorítmico e trilhas imutáveis de auditoria.

- Normalização e Triagem Multivetorial: Viabilizar o processamento técnico de sistemas suspeitos por meio de três métodos distintos (coleta em sistema ativo — *live*, análise de imagens de disco adquiridas previamente ou extração remota autenticada), traduzindo dados brutos de baixo nível em modelos de achados uniformizados e estruturados.
- Validade Jurídica e Aplicabilidade Pericial: Estruturar saídas periciais legíveis, conclusivas e metodologicamente transparentes, satisfazendo integralmente aos padrões probatórios exigidos por tribunais judiciais, órgãos fiscalizadores e auditorias corporativas externas.
- Conformidade Normativa Estrita: Alinhar cada estágio do ciclo de desenvolvimento de software e cada rotina de processamento analítico às diretrizes formais das normas de perícia digital, gestão de incidentes e segurança de software.
- Ativo Estratégico de Engenharia de Software: Consolidar-se como uma plataforma *open source* robusta, documentada e modular no GitHub, evidenciando domínio sênior em arquitetura de sistemas, algoritmos forenses aplicados e integração segura de software.

3. FRAMEWORK DE CONFORMIDADE FORENSE E DE SEGURANÇA

A confiabilidade de uma solução forense repousa sobre a aderência comprovável aos referenciais normativos da ciência computacional forense. O desenvolvimento deste projeto segue categoricamente as seguintes normas e marcos regulatórios:

- ISO/IEC 27037:2012 (Identificação, Coleta, Aquisição e Preservação de Evidências Digitais): Estabelece que toda ação do operador sobre o sistema seja minimamente invasiva e estritamente auditada. O toolkit implementa o Manifesto de Evidências estruturado com carimbos de tempo em padrão UTC, registros de identidade do operador, detalhamento do método de captura e cálculo simultâneo de hashes criptográficos.
- ISO/IEC 27041:2015 (Garantia da Adequação de Métodos Forenses): Exige validação metodológica de todas as ferramentas computacionais empregadas. Cada componente interpretador (*parser*) é submetido a baterias de testes automatizados com dados sintéticos e bases forenses de referência chanceladas internacionalmente, atestando ausência de viés e exatidão na extração dos dados.
- ISO/IEC 27042:2015 (Diretrizes para Análise e Interpretação de Evidências Digitais): Impõe o uso de algoritmos analíticos determinísticos (mesmo conjunto de dados de entrada gera rigorosamente o mesmo resultado analítico). Cada achado aponta para seu artefato originário, explicita o grau de confiança da inferência técnica e mapeia os fatos segundo a matriz tática do framework MITRE ATT&CK.

- ISO/IEC 27043:2015 (Princípios e Processos de Investigação de Incidentes): Padroniza o fluxo processual da investigação em etapas formais e prevê o registro inviolável de eventos operacionais executados pela ferramenta, mediante trilhas imutáveis encadeadas criptograficamente.
- ABNT NBR ISO/IEC 27037 e Resolução nº 552/2021 do Conselho Nacional de Justiça (CNJ): Regula a produção e a verificação da prova digital no processo judicial brasileiro. O toolkit viabiliza a verificação cruzada de integridade a qualquer tempo, assegura o contraditório mediante descrição exaustiva e reproduzível da metodologia aplicada e adota terminologia processual conforme ao CPC e CPP.
- NIST Special Publication 800-86 (Guide to Integrating Forensic Techniques into Incident Response): Modela o fluxo de vida operacional da aplicação através dos quadrantes de identificação, preservação, coleta, análise e geração de relatórios técnicos.
- OWASP Application Security Verification Standard (ASVS): Orienta a segurança interna da ferramenta contra ataques diretos, incluindo tratamento seguro de caminhos de arquivos (mitigando *path traversal*), sanitização e validação rigorosa de artefatos malformados e prevenção contra vazamento acidental de credenciais em logs operacionais.

4. ARQUITETURA DE SOFTWARE E ESTRUTURA MODULAR

O projeto adota uma arquitetura em camadas de alta coesão e baixo acoplamento, espelhando os padrões de projeto já estabelecidos com sucesso nos repositórios compliance-audit e pii-guard. A organização interna do pacote `src/forensic_triage/` obedece à seguinte árvore estrutural:

text

```
src/forensic_triage/├── core/│   ├── evidence_manifest.py   # Manifesto unificado e se
```

5. AMBIENTE DE DESENVOLVIMENTO E ESTRATÉGIA DE COMPATIBILIDADE

O ecossistema primário de trabalho do autor é o sistema operacional Windows 11 Home, utilizando o ambiente integrado Visual Studio Code e o interpretador Python 3.14.7 alocado em um ambiente virtual isolado (`.venv`). O projeto adota uma estratégia de engenharia rigorosa para assegurar previsibilidade e estabilidade:

- Gestão de Dependências com Bindings C: O interpretador Python 3.14, por sua natureza recente, pode apresentar limitações quanto à disponibilidade de binários pré-compilados (*wheels*) para bibliotecas forenses legadas que dependem de código nativo em C (tais como `pytsk3`, `python-registry` e `python-evtx`). Para contornar tais cenários sem comprometer o fluxo, a arquitetura do projeto implementa um esquema de abstração de módulos com suporte a ambiente de execução alternativo

via pyenv-win mantendo Python 3.12 como fallback controlado, ao passo que a camada core, analisadores e lógica de negócios operam nativamente em Python 3.14.

- Ambiente de Testes e Virtualização: Devido à ausência de hipervisor nativo na versão Home do sistema operacional, os cenários controlados de comprometimento e os testes de coleta ao vivo (*live collection*) são provisionados por intermédio da plataforma Oracle VM VirtualBox executando instâncias descartáveis de máquinas virtuais Windows.
- Amostras Forenses Certificadas: O desenvolvimento dos *parsers* e analisadores não depende de infraestruturas ativas comprometidas para validação básica. Utilizam-se bases públicas e documentadas, como os corpora de teste do NIST Computer Forensic Reference Data Sets (CFReDS) e as imagens de pesquisa da Digital Corpora, garantindo repetibilidade científica absoluta aos testes automatizados.
- Padronização de Empacotamento: O controle de dependências obedece estritamente às normas modernas da PEP 621 por meio do arquivo pyproject.toml, integrando definições de linters (Ruff/Flake8), tipagem estática (Mypy) e baterias de testes com pytest.

6. INTEGRAÇÃO COM O ECOSISTEMA E VALORIZAÇÃO NO GITHUB

O Forensic Artifact Triage Toolkit é mantido como um repositório autônomo no GitHub, preservando seu posicionamento técnico especializado no domínio de Digital Forensics and Incident Response (DFIR), mas dispondo de integração de dados formalmente documentada com a plataforma compliance-suite.

A integração intersistemas se realiza na borda de notificação de segurança: perante a consumação de um incidente de segurança com violação de dados corporativos ou cadastrais, o laudo técnico emitido pelo toolkit é exportado em formato JSON padronizado. Este payload estruturado é absorvido diretamente pelo compliance-suite, alimentando o registro histórico de incidentes da entidade auditada, recalculando em tempo real o nível de conformidade do cliente no Plano Assure e gerando subsídios fáticos e probatórios indispensáveis para a elaboração do Comunicado de Incidente de Segurança endereçado à Autoridade Nacional de Proteção de Dados (ANPD), atendendo com precisão ao prazo legal prescrito no Artigo 48 da Lei nº 13.709/2018 (LGPD).

No âmbito do portfólio profissional no GitHub, o repositório é dotado de documentação e engenharia em nível corporativo: arquivo README.md exaustivo em padrão internacional (com diagramas operacionais, guia de uso rápido e referências metodológicas), esteiras de Integração Contínua (GitHub Actions) com checagem automática de testes e análise de segurança de dependências (pip-audit), mapeamento explicativo de aderência aos controles da ISO 27037/27042 e lançamentos de versões controlados via GitHub Releases com binários assinados.

7. ROADMAP OPERACIONAL DE SPRINTS

O desenvolvimento técnico do toolkit é organizado em oito sprints sucessivas, estruturadas com entregáveis claros e critérios técnicos de aceitação definidos (*Definition of Done — DoD*):

7.1 Sprint 0 — Fundação Estrutural, Configuração do Ambiente e CI/CD

- Escopo: Criação do repositório versionado no GitHub, instanciação do layout `src/forensic_triage/`, configuração do `pyproject.toml` aderente à PEP 621, parametrização do `.gitignore`, estruturação do ambiente virtual de desenvolvimento, instalação do ferramental base (Typer, Pydantic, Pytest), redação do README inicial de arquitetura e implementação de esteira de integração contínua (GitHub Actions) para execução de linters e testes.
- Critério de Pronto (DoD): Repositório público configurado, pipeline de automação rodando sem advertências no GitHub Actions, execução bem-sucedida do comando de teste unificado e comando `forensic-triage --help` exibindo o painel base via interface de linha de comando.

7.2 Sprint 1 — Núcleo de Integridade, Criptografia e Cadeia de Custódia

- Escopo: Implementação dos módulos contidos em `core/` e `integrity/`. Desenvolvimento do modelo de dados do Manifesto de Evidências conforme ISO 27037, implementação de funções criptográficas de alto desempenho para processamento em blocos contínuos (SHA-256, SHA3-256 e BLAKE3), construção do motor de cadeia de custódia com encadeamento matemático estrito de hashes, geração de logs imutáveis estruturados em JSON à prova de adulteração e modelagem de assinatura digital do perito responsável.
- Critério de Pronto (DoD): Cobertura de testes unitários superior a 90% sobre o motor de custódia, demonstração matemática automatizada de que a manipulação de um único byte em qualquer etapa corrompe a verificação da cadeia e persistência íntegra do log de auditoria.

7.3 Sprint 2 — Mecanismos de Coleta de Artefatos Forenses

- Escopo: Implementação das interfaces abstratas e especializadas de coleta no pacote `collectors/`. Construção do `live_collector.py` para extração direta de artefatos essenciais em sistemas Windows em modo somente-leitura e com mínima invasividade, e do `image_collector.py` para análise de dumps brutos de volumes e imagens forenses. Vinculação automática de cada artefato capturado ao Manifesto de Evidências com cômputo imediato dos valores criptográficos de integridade.
- Critério de Pronto (DoD): Extração bem-sucedida de alvos predeterminados em máquina virtual de ensaio sem alteração indevida de carimbos temporais de acesso aos arquivos originais, acompanhada da emissão do manifesto auditável correlato.

7.4 Sprint 3 — Conversores e Decodificadores de Artefatos do Windows

- Escopo: Desenvolvimento do pacote parsers/. Implementação de decodificadores dedicados para estruturas críticas: arquivos de aceleração de execução (Windows Prefetch), estruturas binárias do registro do Windows (SYSTEM, SOFTWARE e NTUSER.dat via hives locais), logs transacionais de eventos em formato binário (.evtx) e atalhos de sistema (.lnk). Tratamento preventivo de anomalias em arquivos malformados de acordo com os princípios do OWASP ASVS.
- Critério de Pronto (DoD): Validação técnica dos decodificadores contra arquivos extraídos de imagens de referência pública do NIST CReDS e Digital Corpora, com extração fidedigna de metadados operacionais e ausência de falhas críticas de execução.

7.5 Sprint 4 — Analisadores de Vetores de Comprometimento (Analyzers)

- Escopo: Construção do pacote analyzers/ e especialização dos módulos analíticos. Criação do analisador de persistência (focado em chaves Run/RunOnce, tarefas agendadas e serviços do sistema, com mapeamento direto à técnica T1547 do MITRE ATT&CK), analisador de execução de binários (cruzamento de Prefetch, Amcache e Shimcache), analisador de contas de usuário (detecção de criação de contas e escalonamento de privilégios) e analisador de manipulação defensiva do sistema operacional.
- Critério de Pronto (DoD): Produção consistente de instâncias normalizadas da entidade Finding por cada analisador especializado mediante processamento de artefatos contendo evidências de comprometimento previamente semeadas.

7.6 Sprint 5 — Motores de Correlação, Scoring de Risco e Mapeamento Tático

- Escopo: Implementação do pacote scoring/. Desenvolvimento do risk_engine.py aplicando fórmulas determinísticas para cálculo de severidade e impacto, do correlation_engine.py para identificação de correlações complexas entre múltiplos artefatos (ex.: criação anômala de conta seguida de instalação de serviço e execução de binário não assinado) e do módulo mitre_mapper.py para classificação automática de técnicas e táticas adversárias.
- Critério de Pronto (DoD): Demonstração inequívoca de determinismo algorítmico (o mesmo lote de evidências submetido a múltiplas iterações gera identicamente as mesmas pontuações de risco) e correlação cruzada operando eficazmente sobre eventos heterogêneos.

7.7 Sprint 6 — Mecanismos de Geração de Laudos e Relatórios Técnicos

- Escopo: Desenvolvimento das rotinas contidas em reporting/. Criação de gerador de relatórios técnicos interativos em formato HTML enriquecido com painéis gráficos e linha do tempo de eventos, serialização de relatórios estruturados em JSON para alimentação de ferramentas de SIEM e integração com o compliance-suite, consolidação de super-timeline em formato tabular CSV e elaboração do modelo de Laudo Pericial Formal conforme às exigências processuais da Resolução CNJ nº 552/2021.

- Critério de Pronto (DoD): Geração completa do laudo pericial formal e do artefato HTML em cenário experimental de testes, com seções metodológicas completas, transcrição exaustiva da cadeia de custódia e ausência de campos com conteúdo indefinido.

7.8 Sprint 7 — Empacotamento, Ensaios de Conformidade e Distribuição

- Escopo: Compilação da ferramenta como binário autônomo e portátil para ambiente Windows utilizando PyInstaller, eliminando a dependência de interpretadores locais em intervenções de campo; construção de imagem Docker containerizada para execução estável de análises *post-mortem* em servidores; execução da matriz final de ensaios de conformidade contra a ISO 27041; finalização da documentação pericial e publicação da primeira versão estável (v1.0.0) no GitHub Releases acompanhada de checksums de integridade.
- Critério de Pronto (DoD): Executável autônomo validado e funcional em ambiente limpo Windows 11, imagens Docker homologadas, repositório completamente documentado e release público devidamente publicado.

8. DEFINIÇÃO GLOBAL DE PRONTO (DEFINITION OF DONE)

Uma funcionalidade, módulo ou sprint somente será considerada formalmente concluída quando satisfizer plenamente, de forma cumulativa, aos seguintes parâmetros técnicos de engenharia de software:

- Qualidade de Código e Tipagem: O código-fonte deve estar integralmente em conformidade com as convenções da PEP 8, anotado com tipagem estática rigorosa e aprovado pelos linters estáticos sem emissão de alertas.
- Cobertura de Ensaios: Implementação de testes unitários e de integração por meio da suíte pytest, atingindo cobertura global não inferior a 85% sobre as camadas de processamento analítico e manipulação de custódia.
- Reprodutibilidade Científica: Demonstração prática de que os métodos de extração e os algoritmos analíticos mantêm integridade estrita, consistência temporal e determinismo, sem introduzir qualquer alteração nos dados periciados.
- Documentação Formal: Cada módulo deve conter suas respectivas *docstrings* descrevendo os parâmetros de entrada, tipos de dados manipulados, saídas produzidas e referências documentais aos formatos de baixo nível analisados.
- Auditoria de Dependências: O repositório deve estar livre de vulnerabilidades de segurança conhecidas, verificado por ferramentas automáticas de análise de integridade de dependências no pipeline de integração.

